Adaptive Model Poisoning Attack 解説

attack.py における実装内容の整理メモ — Fang et al. (USENIX Security 2020) の考え方をベースにした再実装

対象ファイル: **attack.py**　関連: **aggregation.py** / **robust_strategy.py**　作成日: **2026-08-22**

1. **この攻撃の位置づけ**　2. **単純な攻撃との違い**　3. **動作フロー（なぜサーバ側か）**　4. **攻撃方向の決定**
5. **Median / Trimmed Mean への攻撃**　6. **Krum / Bulyan への攻撃**　7. **ディスパッチと分岐**　8. **「最適な攻撃」なのか？**

1 この攻撃の位置づけ

`attack.py` は Federated Learning (FL) に対する **Model Poisoning Attack**（モデル汚染攻撃）を実装したモジュールである。悪性クライアントがローカル学習後の更新（モデルパラメータ）そのものを細工し、サーバ側の集約結果を意図した方向へ歪めることを目的とする。

その中でも `adaptive_attack` は、**Fang らが提案した Adaptive (Local) Model Poisoning Attack** の考え方を踏襲した実装である。「Adaptive」とは、攻撃者がサーバの集約手法（Median / Trimmed Mean / Krum / Bulyan など）を事前に把握しており、その手法の弱点に合わせて悪性ベクトルを都度作り替える、という意味である。

2 単純な攻撃との違い

`attack.py` には性質の異なる2種類の攻撃が同居している。

種類	関数	攻撃者が持つ知識	特徴
単純な攻撃 (更新量ベース)	`sign_flip_delta` `scale_delta`	自分自身の更新のみ	Δ（ローカル更新量）の符号を反転する／定数倍に拡大するだけ。サーバの集約手法は一切考慮しない。クライアント側（`client_app.py`）で完結する。
Adaptive 攻撃	`adaptive_attack`	**全クライアントの更新 + サーバの集約手法**（論文でいう "full knowledge" 攻撃者）	集約手法ごとに悪性ベクトルの作り方を変える。honestな更新の分布を見て、その分布の境界ギリギリを攻める。

3 動作フロー：なぜサーバ側で実行するのか

Flower のクライアントプロセスは互いに独立しており、あるクライアントが他クライアントの更新を覗き見ることはできない。しかし adaptive attack の前提（他クライアントの更新も把握している「結託した悪性クライアント」）を素直に満たせるのはサーバ側だけである。

そのため `adaptive_attack` は、サーバ側の `robust_strategy.py` にある `aggregate_train` から、次のタイミングで呼び出される。



図1: 集約前に悪性クライアント分のベクトルだけを攻撃済みベクトルに差し替える

実装上は `aggregation.py` と同様、torch非依存の `numpy` 2次元配列 `(n_clients, dim)` を扱う純粋関数として書かれている。honestなベクトルはそのままに、`malicious_idx` に該当する行だけを書き換えて返す。

4 攻撃方向の決定：`_attack_direction`

すべての攻撃に共通する第一歩が「攻撃方向 `s`」の決定である。

```
s = sign(global_vector − mean(honestな更新))
```

honestなクライアントの更新の平均が向かおうとしている方向と**逆向き**に、グローバルモデルを押しやる方向を求めている。符号が0になった座標（方向が定まらない）は正方向に決め打ちする（0のままだと攻撃効果が出ないため）。

> **POINT**
> 学習が正しく進んでいれば honest な更新は「グローバルモデルを改善する方向」に動く。攻撃者はその方向を推定し、真逆に押し返すことで学習を妨害する。

5 Median / Trimmed Mean への攻撃 境界値攻撃

Median や Trimmed Mean は「座標ごとに極端な値を除外・無視する」性質を持つ防御手法である。そこで攻撃側は、**座標ごとの honest な最大値・最小値のさらに外側**に悪性値を割り当てる。

```
level      = strength * (rank + 1) / n_malicious
push_up    = w_max + level * spread   # 攻撃方向が正の座標
push_down  = w_min - level * spread   # 攻撃方向が負の座標
```



図2: honestな範囲の外側に、悪性クライアント間で押し出し量をずらして配置する

もし全悪性クライアントを同じ極端な値にすると、Trimmed Mean のtrim処理でまとめて丸ごと除外されてしまう。これを避けるため、悪性クライアントごとに `rank`（何番目か）に応じて押し出し具合を少しずつ変える＝**stagger**させている。

6 Krum / Bulyan への攻撃 数値探索で最適化

Krum は「他の更新との距離が最も近い1点」を選ぶ防御手法である（Bulyan は内部で反復的にKrum選択を行う）。この性質を逆手に取り、**悪性クライアント全員を1点に密集させて** honest クラスタのすぐ外側に配置する。

```
crafted_malicious = honest_center + λ * s
```

- **起点は honest クラスタの中心**（前ラウンドの global ではない）。1ラウンドの学習で honest クラスタ自体が global から離れて動くケースを考慮し、常に「今の honest 集団」を基準にする。
- λ が小さすぎる → honest に近すぎて攻撃効果が薄い。
- λ が大きすぎる → honest から離れすぎて Krum に選ばれなくなる。

λの探索アルゴリズム

① 範囲拡大フェーズ
初期値 `lambda_init`（honest更新同士の典型距離の2倍）から出発し、Krumに選ばれ続ける限り λ を倍々に増やして探索範囲の上限を見つける。

② 二分探索フェーズ
「選ばれる最大の λ」を二分探索で絞り込む。選ばれなければ探索幅を狭め、選ばれれば広げる、を既定回数繰り返す。

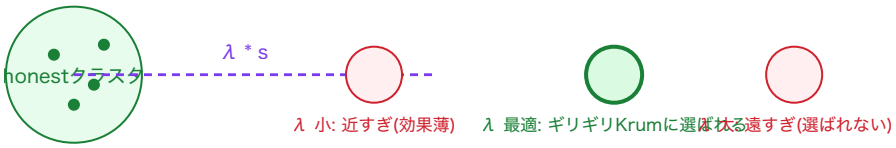


図3: honestクラスタ中心から方向 s へ λ を伸ばし、Krumに選ばれる最大の λ を二分探索する

> **フォールバック**
> どの λ でも悪性クライアントが Krum に選ばれない場合は、攻撃を諦めて honest な更新をそのまま返す（`vectors.copy()`）。

7 ディスパッチと分岐：`adaptive_attack`

`adaptive_attack` は集約手法名の文字列を見て、上記2種類の攻撃のどちらを使うか振り分けるだけのディスパッチ関数である。

aggregation_method	呼び出す攻撃	理由
`median` / `trimmed_mean`	`_craft_trimmed_mean_median_attack`	座標ごとの境界値攻撃が有効
`krum` / `bulyan`	`_craft_krum_attack`	密集＋距離最大化が有効
`fedavg` など	攻撃せずそのまま返す	単純平均はrobustでないため、そもそも `sign_flip` / `scale` で十分効く。adaptive攻撃の対象外

また悪性クライアントが0人、あるいはhonestなクライアントが0人の場合も、そのまま `vectors.copy()` を返す。

8 「最適な攻撃をしている」のか？

ご質問の「最適な攻撃をしているのか」という点について、対象手法ごとに性質が異なる。

対象	最適化の度合い	説明
Krum / Bulyan	数値的に最適化	「Krumに選ばれる範囲でλ（honestクラスタからの距離）を最大化する」という1次元の最適化問題を、範囲拡大＋二分探索で数値的に解いている。局所的な意味では最適解に近い値を求めている。
Median / Trimmed Mean	ヒューリスティック	数理最適化ではなく、「honestな範囲の外側に段階的にずらして配置する」という設計的なルールベースの攻撃。効果はあるが、最適解を数値的に探索しているわけではない。

> **原論文との対応について（実装コメントより）**
>
> 本実装は Fang et al. (USENIX Security 2020) が示す「集約手法の特性を突いた最適化ベースの攻撃」という**中核アイデアを踏襲した独自実装**であり、論文本文の数式をそのまま再現したものではない（近似・簡略化した再実装）。卒論で原論文との厳密な数式対応を主張する場合は、原論文の該当箇所と本実装の差分を別途確認・記載する必要がある。

まとめ： `adaptive_attack` は Model Poisoning Attack の一種であり、「サーバの集約手法をあらかじめ知った上で、その手法の弱点に適応させて攻撃を作る」という意味で adaptive／最適化的である。ただし全ての対象手法で厳密な数理最適解を求めているわけではなく、Krum/Bulyanでは数値探索による最適化、Median/Trimmed Meanでは設計的なヒューリスティックという2段構えの実装になっている。